

AI-Native Engineer Capstone

ClaimFlow — Claims Intake, Assessment & Settlement Platform

Business Case BC-AINE-005 · Domain: Insurance — Claims Operations

1. Project Identity

Business Case Title	Claims Intake, Assessment & Settlement Platform
Business Case ID	BC-AINE-005
Domain	Insurance — Claims Operations
Project Type	AI-Native Engineering Capstone
Cohort	(filled by Operations team)

2. Engagement Overview

Duration	20 hours
Format	Individual
Evaluation Mode	Automated review of submitted Git repository against the AI-Native Engineering Rubric (21 parameters / 100 marks across 5 categories)
Submission	Push final code to your assigned Virtusa GitLab repository by the cohort cut-off
Review Output	Per-learner Excel report (Summary, Categories, Scorecard, Detailed, Improvement)
Grade Bands	Distinction ≥ 90 · Merit 75–89 · Pass 60–74 · Not Yet Passed < 60

What is evaluated: AI-native engineering practices (specs, agents, skills, hooks, CLAUDE.md hierarchy, plugin packaging, MCP, Harness plugin usage), business understanding (context, rules, AC), UI architectural discipline (structural rules, TDD), technical implementation (code, tests, coverage, UI readiness), and CI/CD integration (PR-driven workflows, Claude in CI).

What is not evaluated: the specific technology stack chosen, the visual polish of the UI, or the use of any AI provider beyond Claude Code.

3. Problem Statement & Expected Solution

3.1 Problem

Horizon Insurance needs to streamline claims processing across motor, health, and life products. You build the FNOL-to-settlement pipeline — initiate, document checks, fraud screening, assessment, decision, and payout — under one constraint: no manual coding. Every line of production code must be generated by Claude Code agents under your supervision.

3.2 Your Role

AI-Native Engineer — architect, supervisor, and quality gatekeeper. You write specifications, configure agents, install guardrails, and review pull requests. You do not write production code by hand.

3.3 Expected Solution

A working platform delivered as a Git repository that demonstrates:

- A complete AI-native engineering substrate built on the Claude Harness Engine plugin foundation, extended with claims-operations-specific specs, agents, skills, hooks, commands, and MCP integration.
- A functional claims management platform built end-to-end by Claude Code agents, with adjudication and settlement workflows for at least 3 claim types and no hand-written production code.
- An evidence trail (PRs, agent logs, post-mortems, sprint contracts) proving the work was agent-led.

3.4 Applicable Rules

No-Hand-Coding Rule: All production code, tests, and migrations must be generated by Claude Code agents. Hand-edits are limited to specs, CLAUDE.md, agent definitions, hooks, skills, and other substrate files.

Spec-Is-Truth Rule: Whenever a spec and code disagree, the spec wins. Update the spec deliberately and regenerate the code.

PR-Only Merge Rule: No direct commits to main. All changes flow through agent-reviewed pull requests.

Synthetic-Data Rule: Use tiny dummy data. Do not use Virtusa confidential or customer-sensitive data for this project.

4. Technology Stack

Choose any stack that matches your background. The stack does not affect evaluation.

Layer	Options
Frontend	React · Angular · Vue · server-rendered (Thymeleaf / Razor)
Backend	Java + Spring Boot · .NET + ASP.NET Core Web API · Python + FastAPI · Node.js + Express (fastify)
Database	In-memory (H2 / SQLite) · RDBMS (MySQL / MariaDB / SQL Server) · NoSQL (MongoDB / DynamoDB)
Build & Dependency Management	Maven / Gradle · npm / yarn · Poetry / pipenv

Testing	JUnit / NUnit / pytest / Jest · Playwright (E2E) · ArchiUnit / dependency-cruiser / import-linter
---------	---

5. Acceptance Criteria & Non-Functional Requirements

5.1 Functional Acceptance Criteria

ID	Criterion
AC-01	Customer can file a First Notice of Loss (FNOL) with policy number, incident date, claim type, and supporting documents (stubbed upload)
AC-02	Claim is rejected with PolicyNotActiveException when the linked policy is not ACTIVE on the incident date
AC-03	Document-checklist enforcement varies by claim type (motor: police FIR + invoice; health: hospital bill + discharge summary; life: death certificate)
AC-04	Fraud screening returns a deterministic score; claim is flagged when score exceeds the configured threshold
AC-05	Assessment computes payable_amount = min(claim_amount, sum_insured) – deductibles – co_pay; invariant: payable_amount ≥ 0
AC-06	Decision returns AUTO_APPROVE / MANUAL_REVIEW / REJECT with reason codes
AC-07	Settlement generates an immutable payout record linked to the claim
AC-08	Reopen creates a new sub-claim linked to the original
AC-09	Admin can override decisions with command and reason code
AC-10	Claim state transitions are enforced; invalid transitions raise InvalidClaimStateException

Note. Each AC must have at least one test referencing its AC-N identifier.

5.2 Non-Functional Requirements

ID	Requirement
NFR-01	Payout, deductible, and co-pay amounts are computed in fixed-point (BigDecimal / decimal)
NFR-02	Claim records, fraud scores, assessments, and settlements are append-only
NFR-03	PII (health data, claim narratives, document content) is never written to logs
NFR-04	Authentication boundary enforced at controller

	layer
NFR-05	Database migrations are append-only
NFR-06	Structured JSON logs with request correlation IDs
NFR-07	Health endpoint returns 200 within 1 second of successful startup
NFR-08	Architecture rules enforced as automated tests

6. Functional Scope

6.1 Customer / End-User Functionality

- File FNOL: Submit claim with policy number, incident details, claim type, and supporting documents (stubbed)
- Track: View claim status, decision, reason codes, and outstanding document requirements
- Dispute: Raise a dispute on a settled claim; original moves to REOPENED state with a new sub-claim

6.2 Internal & System Functionality

- Document Verification Queue: Per-claim-type checklist; mark each document VERIFIED or MISSING
- Fraud Screening Engine: Rule-based pattern match; flag claims that exceed the configured threshold
- Assessor Workbench: Review claim, fraud flags, assessed amount, decision rationale; submit decision
- Settlement Module: Generate payout records; trigger stubbed payment to the customer's account
- Admin Dashboard: Claim queue by status and product, fraud alert queue, payout audit trail

6.3 Out of Scope

You do not need to build:

- Real payment orchestration
- Real ML-based fraud detection (rule-based stub only)
- Real payment rails for payout disbursement

7. AI-Native Implementation Expectations

Foundation: Install the Claude Harness Engine plugin first. The harness ships 7 base agents. On top of that foundation, your project-specific substrate is listed in this section.

7.1 Claude Code Specs (40 marks)

- specs/app_spec.md (root spec) and specs/<feature>_spec.md per major feature
- Layered CLAUDE.md — root + per-module + per-feature_AGENTS.md

- At least 3 project-specific skills under `claude/skills/`
- At least 2 project-specific custom commands under `claude/commands/`
- At least 2 project-specific hooks under `claude/hooks/`
- At least 3 project-specific agents under `claude/agents/`
- At least one project-specific debugging log or `pg.json`

7.2 Business Understanding (20 marks)

- `docs/business-case.md` covering product value proposition
- At least 6 rule/policy statements embedded in `claude.rules/`
- At least one responsive layout (React / Angular / Vue)
- Acceptance criteria expressed in Given-When-Then format

7.3 Architectural Discipline (8 marks)

- Architecture-test directory with at least 3 structural tests
- ArchUnit / dependency-cruiser / import-linter configured
- TDD discipline document at `docs/tdd.md`

7.4 Technical Implementation (26 marks)

- `docs/architecture.md` with at least 1 architecture diagram
- Substantive codebase with at least 3000 lines of generated code
- Playwright for UI validation
- Tests tagged with AC IDs
- At least 20 unit tests with coverage artifact

7.5 CI/CD (6 marks)

- `.gitlab-ci.yml` (or `github/workflows/`) with build and test pipeline
- Claude Code Action wired into CI workflow
- At least 3 PR-driven merges: zero direct commits to main

8. Expected Outcomes & Deliverables

By the end of 20 hours, the submitted Git repository must contain the Mandatory items below.

8.1 Mandatory

1. Working Application: Claims management platform deployable locally via a single command
2. `docs/business-case.md`: The business context document covering problem and value proposition
3. Specification Files: Root spec + per-feature specs + project agent substrate
4. AC-linked Specs: Every Acceptance Criteria with explicit sections
5. Project Agent Substrate: `claude/agents/`, `claude/skills/`, `claude/commands/`, `claude/hooks/`
6. Claude Harness Engine Integration: Plugin installed with at least 3 PR-driven merges
7. Manifest: `mcp.json` with Playwright MCP active

8.2 Good-to-Have

- Architecture & TDD Documentation with diagrams
- Automation & Evidence demonstrating agent-led bug detection
- Knowledge Deposit Log documenting learnings
- Multiple Sprint Cycles covering different claim types
- Bonus Agents beyond required additions

On completion of the capstone, you will have demonstrated building a working, well-architected claims processing platform without writing code yourself, while maintaining full control through AI-native engineering practices.